

```
1
2
3
4 class AppConstants {
5     // App Info
6     static const String appName = '■■■■';
7     static const String packageName = 'com.molitao.app';
8     static const String version = '1.0.0';
9
10    // Tab Names
11    static const String homeTab = '■■■';
12    static const String chatTab = '■■■';
13    static const String tradingPostTab = '■■■■';
14    static const String contactsTab = '■■■■';
15    static const String profileTab = '■■■■■';
16
17    // Tab Routes
18    static const String homeRoute = '/';
19    static const String chatRoute = '/chat';
20    static const String tradingPostRoute = '/trading-post';
21    static const String contactsRoute = '/contacts';
22    static const String profileRoute = '/profile';
23
24    // Asset Paths
25    static const String tabIconsPath = 'assets/images/tab/';
26    static const String addIconPath = 'assets/images/add.png';
27
28    // Tab Icons
29    static const String homeTabIcon = '${tabIconsPath}tab1.png';
30    static const String homeTabIconActive = '${tabIconsPath}tab1_b.png';
31    static const String chatTabIcon = '${tabIconsPath}tab2.png';
32    static const String chatTabIconActive = '${tabIconsPath}tab2_b.png';
33    static const String contactsTabIcon = '${tabIconsPath}tab3.png';
34    static const String contactsTabIconActive = '${tabIconsPath}tab3_b.png';
35    static const String profileTabIcon = '${tabIconsPath}tab4.png';
36    static const String profileTabIconActive = '${tabIconsPath}tab4_b.png';
37    static const String tradingPostTabIcon = addIconPath;
38 }
39 class StorageKeys {
40     // Authentication
41     static const String token = 'token';
42     static const String refreshToken = 'refresh_token';
43     static const String userId = 'user_id';
44     static const String userData = 'user_data';
45
46     // User Preferences
47     static const String themeMode = 'theme_mode';
48     static const String language = 'language';
49     static const String notificationsEnabled = 'notifications_enabled';
50     static const String lastLoginTime = 'last_login_time';
```

■ 2 ■

```
101     static const TextStyle headline1 = TextStyle(
102     96.0,
103     FontWeight.w300,
104     AppColors.textPrimary,
105     );
106
107
108
109     static const TextStyle headline2 = TextStyle(
110     60.0,
111     FontWeight.w300,
112     AppColors.textPrimary,
113     );
114
115     static const TextStyle headline3 = TextStyle(
116     48.0,
117     FontWeight.w400,
118     AppColors.textPrimary,
119     );
120
121     static const TextStyle headline4 = TextStyle(
122     34.0,
123     FontWeight.w400,
124     AppColors.textPrimary,
125     );
126
127     static const TextStyle headline5 = TextStyle(
128     24.0,
129     FontWeight.w400,
130     AppColors.textPrimary,
131     );
132
133     static const TextStyle headline6 = TextStyle(
134     20.0,
135     FontWeight.w500,
136     AppColors.textPrimary,
137     );
138
139     // Subtitle styles
140     static const TextStyle subtitle1 = TextStyle(
141     16.0,
142     FontWeight.w400,
143     AppColors.textPrimary,
144     );
145
146     static const TextStyle subtitle2 = TextStyle(
147     14.0,
148     FontWeight.w500,
149     AppColors.textPrimary,
150     );
```

```
151
152     // Body styles
153     static const TextStyle bodyText1 = TextStyle(
154     16.0,
155     FontWeight.w400,
156     AppColors.textPrimary,
157     );
158
159
160
161     static const TextStyle bodyText2 = TextStyle(
162     14.0,
163     FontWeight.w400,
164     AppColors.textPrimary,
165     );
166
167     // Button style
168     static const TextStyle button = TextStyle(
169     14.0,
170     FontWeight.w500,
171     AppColors.textPrimary,
172     );
173
174     // Caption style
175     static const TextStyle caption = TextStyle(
176     12.0,
177     FontWeight.w400,
178     AppColors.textSecondary,
179     );
180
181     // Overline style
182     static const TextStyle overline = TextStyle(
183     10.0,
184     FontWeight.w400,
185     AppColors.textSecondary,
186     1.5,
187     );
188 }
189 import 'package:flutter/material.dart';
190 import 'app_colors.dart';
191 import 'app_text_styles.dart';
192
193 class AppTheme {
194     static ThemeData get lightTheme {
195         return ThemeData(
196         true,
197         colorScheme:
198             ColorScheme.fromSeed(
199             AppColors.primary,
200             Brightness.light,
```

```
201         ).copyWith(
202     AppColors.primary,
203     AppColors.primaryLight,
204     AppColors.surface,
205     AppColors.error,
206     ),
207     _buildTextTheme(),
208     _buildAppBarTheme(),
209     _buildBottomNavigationBarTheme(),
210
211
212     _buildElevatedButtonTheme(),
213     _buildOutlinedButtonTheme(),
214     );
215   }
216
217   static TextTheme _buildTextTheme() {
218     return const TextTheme(
219     AppTextStyles.headline1,
220     AppTextStyles.headline2,
221     AppTextStyles.headline3,
222     AppTextStyles.headline4,
223     AppTextStyles.headline5,
224     AppTextStyles.headline6,
225     AppTextStyles.bodyText1,
226     AppTextStyles.bodyText2,
227     AppTextStyles.button,
228     AppTextStyles.caption,
229     AppTextStyles.overline,
230     );
231   }
232
233   static AppBarTheme _buildAppBarTheme() {
234     return const AppBarTheme(
235     AppColors.surface,
236     AppColors.textPrimary,
237     0,
238     true,
239     AppTextStyles.headline6,
240     );
241   }
242
243   static BottomNavigationBarThemeData _buildBottomNavigationBarTheme() {
244     return BottomNavigationBarThemeData(
245     AppColors.surface,
246     AppColors.primary,
247     AppColors.textSecondary,
248     AppTextStyles.caption,
249     AppTextStyles.caption,
250     BottomNavigationBarType.fixed,
```

```
251     );
252   }
253
254   static ElevatedButtonThemeData _buildElevatedButtonTheme() {
255     return ElevatedButtonThemeData(
256       style:
257         ElevatedButton.styleFrom(
258   AppColors.primary,
259   Colors.white,
260   RoundedRectangleBorder(
261   BorderRadius.circular(8.0),
262
263
264         ),
265     const EdgeInsets.symmetric(
266     12.0,
267     24.0,
268
269     ),
269     ).copyWith(
270   WidgetStateProperty.all(
271   FontWeight.bold),
272
273     ),
274   );
275   }
276
277   static OutlinedButtonThemeData _buildOutlinedButtonTheme() {
278     return OutlinedButtonThemeData(
279       style:
280         OutlinedButton.styleFrom(
281   AppColors.primary,
282   const BorderSide(color: AppColors.primary),
283   RoundedRectangleBorder(
284   BorderRadius.circular(8.0),
285
286         ),
286     const EdgeInsets.symmetric(
287     12.0,
288     24.0,
289
290         ),
290     ).copyWith(
291   WidgetStateProperty.all(
292   FontWeight.bold),
293
294     ),
295   );
296   }
297 }
298 import 'package:flutter/material.dart';
299
300 class AppColors {
```

```
301    // ■■■ - ■■
302    static const Color primary = Color(0xffff4835a);
303    static const Color primaryLight = Color(0xffff7a17f);
304    static const Color primaryDark = Color(0xffd06a43);
305
306    // ■■■
307    static const Color background = Color(0xffff5f5f5);
308    static const Color surface = Color(0xfffffffff);
309    static const Color textPrimary = Color(0xff212121);
310    static const Color textSecondary = Color(0xff757575);
311    static const Color textHint = Color(0xff9e9e9e);
312    static const Color divider = Color(0xffe0e0e0);
313
314
315
316    // ■■■
317    static const Color success = Color(0xff4caf50);
318    static const Color warning = Color(0xffffc107);
319    static const Color error = Color(0xffff44336);
320    static const Color info = Color(0xff2196f3);
321  }
322  import 'package:flutter/material.dart';
323  import 'package:flutter/services.dart';
324
325  /// ■■■■■■■■
326  ///
327  /// ■■■■■■■■■■■■
328  /// - ■■■■■■
329  /// - ■■■■■■
330  /// - ■■■■■■■■■■■■■■
331  /// - ■■■■
332  class ChatInputBar extends StatefulWidget {
333    final Function(String) onSendText;
334    final VoidCallback? onEmojiTap;
335    final VoidCallback? onImagePick;
336    final VoidCallback? onFilePick;
337    final VoidCallback? onMoreTap;
338    final String? placeholder;
339    final int maxLength;
340
341    const ChatInputBar({
342      super.key,
343      required this.onSendText,
344      this.onEmojiTap,
345      this.onImagePick,
346      this.onFilePick,
347      this.onMoreTap,
348      this.placeholder,
349      this.maxLength = 700,
350    });
```

```
351
352     @override
353     State<ChatInputBar> createState() => _ChatInputBarState();
354 }
355
356 class _ChatInputBarState extends State<ChatInputBar> {
357     final TextEditingController _controller = TextEditingController();
358     final FocusNode _focusNode = FocusNode();
359     bool _hasText = false;
360
361     @override
362     void initState() {
363         super.initState();
364         _controller.addListener(_onTextChanged);
365     }
366
367
368
369     @override
370     void dispose() {
371         _controller.removeListener(_onTextChanged);
372         _controller.dispose();
373         _focusNode.dispose();
374         super.dispose();
375     }
376
377     void _onTextChanged() {
378         setState(() {
379             _hasText = _controller.text.isNotEmpty;
380         });
381     }
382
383     void _handleSend() {
384         final text = _controller.text.trim();
385         if (text.isNotEmpty) {
386             widget.onSendText(text);
387             _controller.clear();
388         }
389     }
390
391     @override
392     Widget build(BuildContext context) {
393         return Container(
394             BoxDecoration(
395                 Colors.white,
396                 [
397                     BoxShadow(
398                         Colors.black.withOpacity(0.05),
399                         4,
400                         const Offset(0, -1),
```



```
401         ),
402       ],
403     ),
404     SafeArea(
405       Padding(
406         const EdgeInsets.symmetric(horizontal: 8, vertical: 8),
407         Row(
408           [
409             // ■■■■
410             _buildIconButton(
411               Icons.emoji_emotions_outlined,
412               widget.onEmojiTap,
413             ),
414             // ■■■
415             Expanded(
416               Container(
417                 const BoxConstraints(maxHeight: 100),
418                 BoxDecoration(
419                   Colors.grey.shade100,
420                   BorderRadius.circular(20),
421                 ),
422                 TextField(
423                   _controller,
424                   _focusNode,
425                   null,
426                   widget.maxLength,
427                   InputDecoration(
428                     widget.placeholder ?? '■■■■',
429                     TextStyle(
430                       Colors.grey.shade400,
431                       15,
432                     ),
433                   ),
434                   InputBorder.none,
435                   const EdgeInsets.symmetric(
436                     16,
437                     10,
438                   ),
439                   '', // ■■■■■■
440                 ),
441                 const TextStyle(fontSize: 15),
442                 TextInputAction.send,
443                 (_) => _handleSend(),
444               ),
445             ),
446           ],
447         ),
448       ),
449     ),
450     8),
```

■10■

```
501     }
502   }
503
504   /// ■■■■■■
505   class MoreActionsPanel extends StatelessWidget {
506     final VoidCallback? onImagePick;
507     final VoidCallback? onFilePick;
508     final VoidCallback? onCameraPick;
509
510     const MoreActionsPanel({
511       super.key,
512       this.onImagePick,
513       this.onFilePick,
514       this.onCameraPick,
515     });
516
517     @override
518     Widget build(BuildContext context) {
519       return Container(
520         120,
521         const EdgeInsets.symmetric(horizontal: 20, vertical: 16),
522
523         BoxDecoration(
524           Colors.grey.shade50,
525           Border(top: BorderSide(color: Colors.grey.shade200)),
526         ),
527         Row(
528           MainAxisAlignment.spaceAround,
529           [
530             Icons.image, label: '■■', onTap: onImagePick),
531             _buildActionItem(
532               Icons.camera_alt,
533               '■■',
534               onCameraPick,
535             ),
536             _buildActionItem(
537               Icons.insert_drive_file,
538               '■■',
539               onFilePick,
540             ),
541           ],
542         ),
543       );
544     }
545
546     Widget _buildActionItem({
547       required IconData icon,
548       required String label,
549       VoidCallback? onTap,
550     }) {
```

```
551    }) {
552      return GestureDetector(
553        onTap,
554        Column(
555          MainAxisSize.min,
556          [
557            Container(
558              50,
559              50,
560              BoxDecoration(
561                Colors.white,
562                BorderRadius.circular(10),
563                [
564                  BoxShadow(
565                    Colors.black.withOpacity(0.05),
566                    4,
567                    const Offset(0, 1),
568                    ),
569                ],
570              ),
571              Icon(icon, color: const Color(0xFFf4835a), size: 28),
572              ),
573              8),
574
575              Text(
576                label,
577                TextStyle(fontSize: 12, color: Colors.grey.shade700),
578              ),
579            ],
580          ),
581        );
582      }
583    }
584  }
585  import 'package:flutter_riverpod/flutter_riverpod.dart';
586  import 'package:go_router/go_router.dart';
587  import '../presentation/pages/tabbar/main_tab_page.dart';
588  import '../presentation/pages/home/home_page.dart';
589  import '../presentation/pages/chat/chat_list_page.dart';
590  import '../presentation/pages/chat/group_chat_page.dart';
591  import '../presentation/pages/chat/private_chat_page.dart';
592  import '../presentation/pages/chat/auction_chat_page.dart';
593  import '../presentation/pages/trading_post/trading_post_page.dart';
594  import '../presentation/pages/trading_post/post_detail_page.dart';
595  import '../presentation/pages/trading_post/add_post_page.dart';
596  import '../presentation/pages/contacts/contacts_page.dart';
597  import '../presentation/pages/profile/profile_page.dart';
598  import '../presentation/pages/user/deposit_log_page.dart';
599  import '../presentation/pages/user/user_info_page.dart';
600  import '../presentation/pages/user/balance_log_page.dart';
```

```

6001 import '../..presentation/pages/user/auction_success_list_page.dart';
6002 import '../..presentation/pages/announce/announce_list_page.dart';
6003 import '../..presentation/pages/auth/login_page.dart';
6004
6005 import '../..presentation/providers/auth_notifier.dart';
6006
6007 /// ██████████
6008 const List<String> _protectedRoutes = [
6009   '/chat',
6010   '/trading-post',
6011   '/contacts',
6012   '/profile',
6013 ];
6014
6015 /// ██████████
6016 bool _requiresAuth(String location) {
6017   return _protectedRoutes.any((route) => location.startsWith(route));
6018 }
6019
6020 /// ■■ GoRouter
6021 GoRouter _createRouter(AuthNotifier authNotifier) {
6022   return GoRouter(
6023     '/home',
6024     authNotifier,
6025     (context, state) {
6026
6027       final isLoggedIn = authNotifier.isLoggedIn;
6028       final location = state.uri.path;
6029
6030       // ████████████████████████████████
6031       if (_requiresAuth(location) && !isLoggedIn) {
6032         return '/login?redirect=${Uri.encodeComponent(location)}';
6033       }
6034
6035       // ████████████████████████████████
6036       if (isLoggedIn && location == '/login') {
6037         return '/home';
6038       }
6039
6040       return null; // ████████
6041     },
6042     [
6043       GoRoute(
6044         '/login',
6045         'login',
6046         (context, state) {
6047           final redirect = state.uri.queryParameters['redirect'];
6048           redirect);
6049       },
6050     ],

```

```

651         ),
652         StatefulShellRoute.indexedStack(
653 (context, state, navigationShell) =>
654 navigationShell),
655 [
656         StatefulShellBranch(
657 [
658             GoRoute(
659 '/home',
660 'home',
661 (context, state) => const HomePage(),
662             ),
663         ],
664         ),
665         StatefulShellBranch(
666 [
667             GoRoute(
668 '/chat',
669 'chat',
670 (context, state) => const ChatListPage(),
671             ),
672         ],
673         ),
674         StatefulShellBranch(
675 [
676             GoRoute(
677 '/trading-post',
678
679
680 'trading-post',
681 (context, state) => const TradingPostPage(),
682 [
683             GoRoute(
684 'add',
685 'add-post',
686 (context, state) => const AddPostPage(),
687             ),
688             GoRoute(
689 'detail/:id',
690 'post-detail',
691 (context, state) {
692                 final idStr = state.pathParameters['id'] ?? '0';
693                 final id = int.tryParse(idStr) ?? 0;
694             id);
695             },
696         ],
697         ),
698     ],
699 ],
700 ),

```

```

701         StatefulShellBranch(
702     [
703         GoRoute(
704     '/contacts',
705     'contacts',
706     (context, state) => const ContactsPage(),
707         ),
708     ],
709     ),
710     StatefulShellBranch(
711     [
712         GoRoute(
713     '/profile',
714     'profile',
715     (context, state) => const ProfilePage(),
716     [
717         GoRoute(
718     'deposit-log',
719     'deposit-log',
720     (context, state) => const DepositLogPage(),
721         ),
722         GoRoute(
723     'user-info',
724     'user-info',
725     (context, state) {
726         final extra = state.extra as Map<String, dynamic>;
727         final userId = extra?['userId'] as int?;
728         userId);
729     },
730
731
732     ),
733     GoRoute(
734     'balance-log',
735     'balance-log',
736     (context, state) => const BalanceLogPage(),
737         ),
738     GoRoute(
739     'announce',
740     'announce',
741     (context, state) {
742         final categoryId =
743             state.uri.queryParameters['categoryId'];
744         return AnnounceListPage(
745     categoryId != null
746             ? int.tryParse(categoryId)
747     null,
748         );
749     },
750     ),

```

```
751             GoRoute(  
752 'auction-success-list',  
753 'auction-success-list',  
754 (context, state) => const AuctionSuccessListPage(),  
755     ),  
756     ],  
757     ),  
758     ],  
759     ),  
760     ],  
761     ),  
762     // ■■■■■■  
763     GoRoute(  
764 '/chat/auction',  
765 'chat-auction',  
766 (context, state) => const AuctionChatPage(),  
767     ),  
768     GoRoute(  
769 '/chat/group/:id',  
770 'chat-group',  
771 (context, state) {  
772     final idStr = state.pathParameters['id'] ?? '0';  
773     final id = int.tryParse(idStr) ?? 0;  
774     final name = state.uri.queryParameters['name'] ?? '■■';  
775     return GroupChatPage(  
776 '${id}_group',  
777 id,  
778 name,  
779     );  
780     },  
781     ),  
782  
783  
784     GoRoute(  
785 '/chat/private/:id',  
786 'chat-private',  
787 (context, state) {  
788     final idStr = state.pathParameters['id'] ?? '0';  
789     final id = int.tryParse(idStr) ?? 0;  
790     final name = state.uri.queryParameters['name'] ?? '■■';  
791     final avatar = state.uri.queryParameters['avatar'];  
792     return PrivateChatPage(  
793 id,  
794 name,  
795 avatar,  
796     );  
797     },  
798     ),  
799     ],  
800 );
```



```
801 }
802
803 /// Router Provider
804 final routerProvider = Provider<GoRouter>((ref) {
805     final authNotifier = ref.watch(authNotifierProvider);
806     return _createRouter(authNotifier);
807 });
808 // ■■■URL■■■■
809 // ■ UniApp imageUrlConverter.ts ■■■■
810
811 /// ■■■■URL
812 /// ■ cdn.molitao.top ■■■ image.molitao.top
813 String convertImageUrl(String? url) {
814     if (url == null || url.isEmpty) return '';
815
816     // ■ cdn.molitao.top ■■■ image.molitao.top
817     return url.replaceAll(RegExp(r'https?://cdn\.molitao\.top'), 'https://i...');
818 }
819
820 /// ■■■■■■URL■■
821 List<String> convertImageUrls(List<String>? urls) {
822     if (urls == null || urls.isEmpty) return [];
823     return urls.map((url) => convertImageUrl(url)).toList();
824 }
825
826 /// ■■HTML■■■■URL
827 /// ■■ img ■■■■ src ■ data-url ■■
828 String convertHtmlImageUrls(String html) {
829     if (html.isEmpty) return html;
830
831     var result = html;
832
833     // ■■ data-url ■■■■URL (■■■)
834
835
836     result = result.replaceAllMapped(
837         RegExp(r'data-url="([^\"]*)"'),
838         (match) {
839             final url = match.group(1);
840             if (url != null) {
841                 final converted = convertImageUrl(url);
842                 return 'data-url="$converted"';
843             }
844             return match.group(0)!;
845         },
846     );
847
848     // ■■ data-url ■■■■URL (■■■)
849     result = result.replaceAllMapped(
850         RegExp(r'data-url=\'([^\']*)\''),
```

```
851     (match) {
852         final url = match.group(1);
853         if (url != null) {
854             final converted = convertImageUrl(url);
855             return "data-url='$converted'";
856         }
857         return match.group(0)!;
858     },
859 );
860
861 // ■■ src ■■■■URL (■■■■)■■■■■ !w300 ■■■■■
862 result = result.replaceAllMapped(
863     RegExp(r'src="([^\"]*)"'),
864     (match) {
865         final url = match.group(1);
866         if (url != null) {
867             // ■■ !w300 ■■■■■
868             final cleanUrl = url.replaceAll(RegExp(r'!w300$'), '');
869             final converted = convertImageUrl(cleanUrl);
870             return 'src="$converted"';
871         }
872         return match.group(0)!;
873     },
874 );
875
876 // ■■ src ■■■■URL (■■■■)■■■■■ !w300 ■■■■■
877 result = result.replaceAllMapped(
878     RegExp(r"src='([^\']*)'"),
879     (match) {
880         final url = match.group(1);
881         if (url != null) {
882             // ■■ !w300 ■■■■■
883             final cleanUrl = url.replaceAll(RegExp(r'!w300$'), '');
884             final converted = convertImageUrl(cleanUrl);
885             return "src='$converted'";
886         }
887     }
888     return match.group(0)!;
889 },
890 );
891 );
892
893 return result;
894 }
895
896 /// ■HTML■■■■■■■■URL
897 List<String> extractImageUrlsFromHtml(String html) {
898     if (html.isEmpty) return [];
899
900     final List<String> urls = [];
```

```
901
902     // data-url (data-url)
903     false);
904     var matches = dataUrlRegExp.allMatches(html);
905     for (final match in matches) {
906         final url = match.group(1);
907         if (url != null && url.isNotEmpty) {
908             urls.add(convertImageUrl(url));
909         }
910     }
911
912     // data-url (data-url)
913     false);
914     matches = dataUrlRegExp.allMatches(html);
915     for (final match in matches) {
916         final url = match.group(1);
917         if (url != null && url.isNotEmpty) {
918             urls.add(convertImageUrl(url));
919         }
920     }
921
922     // data-url src (data-url src)
923     if (urls.isEmpty) {
924         false);
925         matches = srcRegExp.allMatches(html);
926         for (final match in matches) {
927             final url = match.group(1);
928             if (url != null && url.isNotEmpty) {
929                 // !w300
930                 final cleanUrl = url.replaceAll(RegExp(r'!w300$'), '');
931                 urls.add(convertImageUrl(cleanUrl));
932             }
933         }
934
935         // src (src)
936         false);
937         matches = srcRegExp.allMatches(html);
938
939
940         for (final match in matches) {
941             final url = match.group(1);
942             if (url != null && url.isNotEmpty) {
943                 // !w300
944                 final cleanUrl = url.replaceAll(RegExp(r'!w300$'), '');
945                 urls.add(convertImageUrl(cleanUrl));
946             }
947         }
948     }
949
950     return urls;
```



```

1001  'emoji_38@2x.png',
1002  'emoji_39@2x.png',
1003  'emoji_40@2x.png',
1004  'emoji_41@2x.png',
1005  'emoji_42@2x.png',
1006  'emoji_43@2x.png',
1007  'emoji_44@2x.png',
1008  'emoji_45@2x.png',
1009  'emoji_46@2x.png',
1010  'emoji_47@2x.png',
1011  'emoji_48@2x.png',
1012  'emoji_49@2x.png',
1013  'emoji_50@2x.png',
1014  'emoji_51@2x.png',
1015  'emoji_52@2x.png',
1016  'emoji_53@2x.png',
1017  'emoji_54@2x.png',
1018  'emoji_55@2x.png',
1019  'emoji_56@2x.png',
1020  'emoji_57@2x.png',
1021  'emoji_58@2x.png',
1022  'emoji_59@2x.png',
1023  'emoji_60@2x.png',
1024  'emoji_61@2x.png',
1025  'emoji_62@2x.png',
1026  'emoji_63@2x.png',
1027  'emoji_64@2x.png',
1028  'emoji_65@2x.png',
1029  'emoji_66@2x.png',
1030  'emoji_67@2x.png',
1031  'emoji_68@2x.png',
1032  'emoji_69@2x.png',
1033  'emoji_70@2x.png',
1034  'emoji_71@2x.png',
1035  'emoji_72@2x.png',
1036  'emoji_73@2x.png',
1037  'emoji_74@2x.png',
1038  'emoji_75@2x.png',
1039  'emoji_76@2x.png',
1040  'emoji_77@2x.png',
1041  'emoji_78@2x.png',
1042
1043
1044  'emoji_79@2x.png',
1045  'emoji_80@2x.png',
1046  'emoji_81@2x.png',
1047  'emoji_82@2x.png',
1048  'emoji_83@2x.png',
1049  'emoji_84@2x.png',
1050  'emoji_85@2x.png',

```

```

1051 'emoji_86@2x.png',
1052 'emoji_87@2x.png',
1053 'emoji_88@2x.png',
1054 'emoji_89@2x.png',
1055 'emoji_90@2x.png',
1056 'emoji_91@2x.png',
1057 'emoji_92@2x.png',
1058 'emoji_93@2x.png',
1059 'emoji_94@2x.png',
1060 'emoji_95@2x.png',
1061 'emoji_96@2x.png',
1062 'emoji_97@2x.png',
1063 'emoji_98@2x.png',
1064 'emoji_99@2x.png',
1065 'emoji_100@2x.png',
1066 'emoji_101@2x.png',
1067 'emoji_102@2x.png',
1068 'emoji_103@2x.png',
1069 'emoji_104@2x.png',
1070 'emoji_105@2x.png',
1071 'emoji_106@2x.png',
1072 'emoji_107@2x.png',
1073 'emoji_108@2x.png',
1074 'emoji_109@2x.png',
1075 'emoji_110@2x.png',
1076 'emoji_111@2x.png',
1077 'emoji_112@2x.png',
1078 'emoji_113@2x.png',
1079 'emoji_114@2x.png',
1080 'emoji_115@2x.png',
1081 'emoji_116@2x.png',
1082 'emoji_117@2x.png',
1083 'emoji_118@2x.png',
1084 'emoji_119@2x.png',
1085 'emoji_120@2x.png',
1086 'emoji_121@2x.png',
1087 'emoji_122@2x.png',
1088 'emoji_123@2x.png',
1089 'emoji_124@2x.png',
1090 'emoji_125@2x.png',
1091 'emoji_126@2x.png',
1092 'emoji_127@2x.png',
1093 'emoji_128@2x.png',
1094
1095
1096 'emoji_129@2x.png',
1097 'emoji_130@2x.png',
1098 'emoji_131@2x.png',
1099 'emoji_132@2x.png',
1100 'emoji_133@2x.png',

```

```

1101 'emoji_134@2x.png',
1102 'emoji_135@2x.png',
1103 'emoji_136@2x.png',
1104 'emoji_137@2x.png',
1105 'emoji_138@2x.png',
1106 'emoji_139@2x.png',
1107 'emoji_140@2x.png',
1108 'emoji_141@2x.png',
1109 };
1110
1111 /// ██████████
1112 static List<String> get emojiCodes => emojiMap.keys.toList();
1113
1114 /// ████████ URL
1115 static String? getEmojiUrl(String code) {
1116     final fileName = emojiMap[code];
1117     if (fileName == null) return null;
1118     return '$emojiUrl$fileName';
1119 }
1120
1121 /// ████████████████ HTML██████ RichText █ Web█
1122 static String decode(String text) {
1123     String result = text;
1124     emojiMap.forEach((code, fileName) {
1125         final url = '$emojiUrl$fileName';
1126         result = result.replaceAll(
1127             code,
1128             24px; height: 24px; vertical-align: middle;" />',
1129         );
1130     });
1131     return result;
1132 }
1133
1134 /// ████████████████
1135 static bool containsEmoji(String text) {
1136     for (final code in emojiMap.keys) {
1137         if (text.contains(code)) return true;
1138     }
1139     return false;
1140 }
1141
1142 /// ████████████████████████████████████████
1143 static List<TextSegment> parseText(String text) {
1144     final List<TextSegment> segments = [];
1145     int currentIndex = 0;
1146
1147
1148
1149 // ████████████████████████████████████████
1150 final pattern = RegExp(r'\\[([^\]]+\\)]');

```

```
1151     final matches = pattern.allMatches(text);
1152
1153     for (final match in matches) {
1154         // ■■■■■■■■
1155         if (match.start > currentIndex) {
1156             segments.add(
1157                 TextSegment(
1158                     TextSegmentType.text,
1159                     text.substring(currentIndex, match.start),
1160                 ),
1161             );
1162         }
1163
1164         // ■■■■■■■■■■■■
1165         final matchedText = match.group(0) ?? '';
1166         final emojiUrl = getEmojiUrl(matchedText);
1167
1168         if (emojiUrl != null) {
1169             segments.add(
1170                 TextSegment(
1171                     TextSegmentType.emoji,
1172                     matchedText,
1173                     emojiUrl,
1174                 ),
1175             );
1176         } else {
1177             // ■■■■■■■■■■■■■■■■
1178             segments.add(
1179                 TextSegmentType.text, content: matchedText),
1180             );
1181         }
1182
1183         currentIndex = match.end;
1184     }
1185
1186     // ■■■■■■■■
1187     if (currentIndex < text.length) {
1188         segments.add(
1189             TextSegment(
1190                 TextSegmentType.text,
1191                 text.substring(currentIndex),
1192             ),
1193         );
1194     }
1195
1196     return segments;
1197 }
1198
1199
1200 }
```



```
1201
1202 /// ■■■■■■
1203 enum TextSegmentType { text, emoji }
1204
1205 /// ■■■■
1206 class TextSegment {
1207   final TextSegmentType type;
1208   final String content;
1209   final String? emojiUrl;
1210
1211   const TextSegment({required this.type, required this.content, this.emoj...
1212 }
1213 import 'package:flutter/material.dart';
1214 import 'package:flutter_riverpod/flutter_riverpod.dart';
1215 import 'app.dart';
1216 import 'data/services/wechat_service.dart';
1217
1218 void main() async {
1219   WidgetsFlutterBinding.ensureInitialized();
1220
1221   final wechatService = WeChatService();
1222   await wechatService.initialize();
1223
1224   MyApp());
1225 }
1226
1227 class MyApp extends StatelessWidget {
1228   MyApp({super.key});
1229
1230   @override
1231   Widget build(BuildContext context) {
1232     return App();
1233   }
1234 }
1235 import 'package:dio/dio.dart';
1236 import '../api/api_client.dart';
1237 import '../api/api_endpoints.dart';
1238 import '../models/chat_message_model.dart';
1239 import '../models/chat_list_item_model.dart';
1240
1241 class ChatRepository {
1242   final ApiClient _apiClient = ApiClient();
1243
1244   Future<bool> sendChannelMessage({
1245     required String channel,
1246     required String message,
1247     ChatMessageType? type,
1248     int? from,
1249     String? fromName,
1250
```

```
1251
1252     String? avatar,
1253     Map<String, dynamic>? payload,
1254   }) async {
1255     try {
1256       // ■■ ChatMessage ■■■■ UniApp ■■■■
1257       final chatMessage = {
1258 type != null ? _chatMessageTypeToString(type) : 'Text',
1259 channel,
1260 from,
1261 fromName,
1262 avatar,
1263 message,
1264 payload,
1265       };
1266
1267       await _apiClient.dio.post(
1268         ApiEndpoints.sendChannelMsg,
1269 {'from': from ?? 0, 'chan': channel, 'message': chatMessage},
1270       );
1271       return true;
1272     } on DioException catch (e) {
1273   ${e.message}');
1274     }
1275   }
1276
1277   Future<bool> sendDirectMessage({
1278     required int toUserId,
1279     required String message,
1280     ChatMessageType? type,
1281     int? from,
1282     String? fromName,
1283     String? avatar,
1284     Map<String, dynamic>? payload,
1285   }) async {
1286     try {
1287       // ■■ ChatMessage ■■■■ UniApp ■■■■
1288       final chatMessage = {
1289 type != null ? _chatMessageTypeToString(type) : 'Text',
1290 toUserId,
1291 from,
1292 fromName,
1293 avatar,
1294 message,
1295 payload,
1296       };
1297
1298       await _apiClient.dio.post(
1299         ApiEndpoints.sendMsg,
1300 {'from': from ?? 0, 'to': toUserId, 'message': chatMessage},
```

```
1301         );
1302
1303
1304         return true;
1305     } on DioException catch (e) {
1306     ${e.message}');
1307     }
1308 }
1309
1310 Future<Map<String, dynamic>?> preConnect() async {
1311     try {
1312         final response = await _apiClient.dio.post(ApiEndpoints.preConnect);
1313         return response.data as Map<String, dynamic>;
1314     } on DioException catch (e) {
1315     ${e.message}');
1316     }
1317 }
1318
1319 /// ■■■■■■■■■■ - ■ UniApp getChanHistory ■■
1320 Future<List<ChatMessage>> getChannelHistory({
1321     required String channel,
1322     required int lastTime,
1323 }) async {
1324     try {
1325         final response = await _apiClient.dio.get(
1326             ApiEndpoints.getChanHistory,
1327             {'chan': channel, 'lastTime': lastTime},
1328         );
1329
1330         final items = response.data['items'] as List? ?? [];
1331         return items.map((json) => ChatMessage.fromJson(json)).toList();
1332     } on DioException catch (e) {
1333     ${e.message}');
1334     }
1335 }
1336
1337 /// ■■■■■■■■■■ - ■ UniApp getPrivateHistory ■■
1338 Future<List<ChatMessage>> getPrivateHistory({
1339     required int userId,
1340     required int lastTime,
1341 }) async {
1342     try {
1343         final response = await _apiClient.dio.get(
1344             ApiEndpoints.getPrivateHistory,
1345             {'id': userId, 'lastTime': lastTime},
1346         );
1347
1348         final items = response.data['items'] as List? ?? [];
1349         return items.map((json) => ChatMessage.fromJson(json)).toList();
1350     } on DioException catch (e) {
```

```

1351   ${e.message}');
1352   }
1353   }
1354
1355
1356
1357   Future<List<ChatMessage>> getOfflineMessages() async {
1358     try {
1359       final response = await _apiClient.dio.get(ApiEndpoints.offline);
1360     [...] }
1361     final items =
1362       response.data['messages'] as List? ??
1363       response.data['items'] as List? ??
1364       [];
1365     return items.map((json) => ChatMessage.fromJson(json)).toList();
1366   } on DioException catch (e) {
1367     ${e.message}');
1368   }
1369   }
1370
1371   Future<List<String>> getChannels() async {
1372     try {
1373       final response = await _apiClient.dio.post(ApiEndpoints.getChannels);
1374       final items =
1375         response.data['channels'] as List? ??
1376         response.data['items'] as List? ??
1377         [];
1378       return items.map((e) => e.toString()).toList();
1379     } on DioException catch (e) {
1380     ${e.message}');
1381     }
1382   }
1383
1384   Future<bool> backoutMessage(String messageId) async {
1385     try {
1386       await _apiClient.dio.post(
1387         ApiEndpoints.backout,
1388         {'messageId': messageId},
1389       );
1390       return true;
1391     } on DioException catch (e) {
1392     ${e.message}');
1393     }
1394   }
1395
1396   Future<bool> leaveChannel(String channel) async {
1397     try {
1398       await _apiClient.dio.get(
1399         ApiEndpoints.leaveChannel,
1400         {'channel': channel},

```

```
1401         );
1402         return true;
1403     } on DioException catch (e) {
1404         ${e.message}');
1405     }
1406
1407
1408     }
1409
1410     Future<bool> subscribeChannel(String channel) async {
1411         try {
1412             await _apiClient.dio.post(
1413                 ApiEndpoints.subChannel,
1414                 {'channel': channel},
1415             );
1416             return true;
1417         } on DioException catch (e) {
1418             ${e.message}');
1419         }
1420     }
1421
1422     Future<bool> deleteChannel(String channel) async {
1423         try {
1424             await _apiClient.dio.get(
1425                 ApiEndpoints.delChannel,
1426                 {'channel': channel},
1427             );
1428             return true;
1429         } on DioException catch (e) {
1430             ${e.message}');
1431         }
1432     }
1433
1434     Future<bool> banUser({
1435         required int userId,
1436         required String channel,
1437         required int durationMinutes,
1438     }) async {
1439         try {
1440             await _apiClient.dio.post(
1441                 ApiEndpoints.banUser,
1442                 {
1443                     userId,
1444                     channel,
1445                     durationMinutes,
1446                 },
1447             );
1448             return true;
1449         } on DioException catch (e) {
1450             ${e.message}');
```

```

1451     }
1452   }
1453
1454   Future<List<ChatListItem>> getChatList() async {
1455     try {
1456       final response = await _apiClient.dio.get(ApiEndpoints.getChatList);
1457
1458
1459
1460   [...] }
1461     final items = response.data['items'] as List? ?? [];
1462     return items.map((json) => ChatListItem.fromJson(json)).toList();
1463   } on DioException catch (e) {
1464     ${e.message}');
1465   }
1466 }
1467
1468 Future<bool> deleteChatList(int chatId) async {
1469   try {
1470     await _apiClient.dio.get(
1471       ApiEndpoints.deleteChatList,
1472       {'id': chatId},
1473     );
1474     return true;
1475   } on DioException catch (e) {
1476     ${e.message}');
1477   }
1478 }
1479
1480 static String _chatMessageTypeToString(ChatMessageType type) {
1481   switch (type) {
1482     case ChatMessageType.text:
1483       return 'Text';
1484     case ChatMessageType.image:
1485       return 'Image';
1486     case ChatMessageType.file:
1487       return 'File';
1488     case ChatMessageType.receipt:
1489       return 'Receipt';
1490     case ChatMessageType.welcome:
1491       return 'Welcome';
1492     case ChatMessageType.goodbye:
1493       return 'Goodbye';
1494     case ChatMessageType.banUser:
1495       return 'BanUser';
1496     case ChatMessageType.backout:
1497       return 'Backout';
1498     case ChatMessageType.auctionStart:
1499       return 'AuctionStart';
1500     case ChatMessageType.auctionBid:

```

```

1501         return 'AuctionBid';
1502     case ChatMessageType.auctionEnd:
1503         return 'AuctionEnd';
1504     case ChatMessageType.auctionDeal:
1505         return 'AuctionDeal';
1506     case ChatMessageType.error:
1507         return 'Error';
1508     case ChatMessageType.kasecStatusChanged:
1509         return 'KasecStatusChanged';
1510
1511     }
1512 }
1513 }
1514 }
1515 import 'package:dio/dio.dart';
1516 import '../api/api_client.dart';
1517 import '../api/api_endpoints.dart';
1518 import '../models/auction_item_model.dart';
1519 import '../models/bid_history_model.dart';
1520 import '../models/list_result.dart';
1521
1522 class AuctionRepository {
1523     final ApiClient _apiClient = ApiClient();
1524
1525     Future<ListResult<AuctionItemDto>> getPublicAuctionList({
1526         int? skipCount = 0,
1527         int? maxResultCount = 10,
1528         int? status, // Add this
1529     }) async {
1530         try {
1531             final queryParameters = {
1532                 skipCount,
1533                 maxResultCount,
1534             };
1535
1536             // Add status parameter if provided
1537             if (status != null) {
1538                 queryParameters['Status'] = status;
1539             }
1540
1541             final response = await _apiClient.dio.get(
1542                 ApiEndpoints.getPublicAuctionList,
1543                 queryParameters,
1544             );
1545
1546             return ListResult<AuctionItemDto>.fromJson(
1547                 response.data as Map<String, dynamic>,
1548                 (json) => AuctionItemDto.fromJson(json),
1549             );
1550         } on DioException catch (e) {

```

```
1551   ${e.message}');
1552   }
1553 }
1554
1555   Future<AuctionItemDto?> startAuction(int auctionItemId) async {
1556     try {
1557       final response = await _apiClient.dio.get(
1558         ApiEndpoints.startAuction,
1559         {'id': auctionItemId},
1560       );
1561
1562
1563
1564
1565
1566
1567
1568     Widget build(BuildContext context) {
1569       final imageMessage = ImageMessage(
1570         message,
1571         isSelf,
1572         onMessageTap,
1573         onMessageLongPress,
1574         onImageTap,
1575       );
1576
1577       if (isSelf) {
1578         return MessageBubble(
1579           MessagePosition.right,
1580           EdgeInsets.zero,
1581           onMessageTap,
1582           onMessageLongPress,
1583           imageMessage,
1584         );
1585       }
1586
1587       return MessageWithAvatar(
1588         _getAvatarUrl(),
1589         message.fromName,
1590         isSelf,
1591         onAvatarTap,
1592         imageMessage,
1593       );
1594     }
1595
1596     String? _getAvatarUrl() {
1597       if (message.avatar == null) return null;
1598       final avatar = message.avatar!;
1599       if (avatar.startsWith('http')) return avatar;
1600       return 'https://image.molitao.top/$avatar';
```


[illegible]

```
1651 Container(
1652   const EdgeInsets.symmetric(horizontal: 8, vertical: 4),
1653   const BoxDecoration(
1654     Color(0xff22C55E),
1655     BorderRadius.only(
1656       Radius.circular(8),
1657       Radius.circular(8),
1658     ),
1659   ),
1660   const Text(
1661     '■■■■',
1662     TextStyle(
1663       Colors.white,
1664       12,
1665       FontWeight.bold,
1666     ),
1667   ),
1668 ),
1669 ),
1670
1671
1672   // ■■■■
1673   Padding(
1674     const EdgeInsets.fromLTRB(16, 20, 16, 8),
1675     Column(
1676       CrossAxisAlignment.start,
1677       [
1678         // ■■■■
1679         Container(
1680           const EdgeInsets.all(8),
1681           RichText(
1682             TextSpan(
1683               const TextStyle(
1684                 Color(0xff16A34A),
1685                 14,
1686               ),
1687             [
1688               '■■ '),
1689               TextSpan(
1690                 dealUserName,
1691                 const TextStyle(fontWeight: FontWeight.bold),
1692               ),
1693               ' ■■■ '),
1694               TextSpan(
1695                 '■${finalPrice.toStringAsFixed(2)}',
1696                 const TextStyle(
1697                   18,
1698                   Color(0xffEF4444),
1699                 ),
1700               ),

```

```
1701 ' ■■■■'),
1702         ],
1703     ),
1704     ),
1705     ),
1706     // ■■■■
1707     Container(
1708       const EdgeInsets.all(16),
1709       const EdgeInsets.symmetric(vertical: 8),
1710       BoxDecoration(
1711         Border.all(
1712           const Color(0xff22C55E),
1713           2.5,
1714         ),
1715         BorderRadius.circular(12),
1716         const Color(0xffBBF7D0),
1717         [
1718           BoxShadow(
1719             const Color(0x3322C55E), // 20% opacity
1720             8,
1721             const Offset(0, 2),
1722           ),
1723         ],
1724       ),
1725     ],
1726   ),
1727   Text(
1728     name,
1729     const TextStyle(
1730       20,
1731       FontWeight.bold,
1732     ),
1733   ),
1734   ),
1735   // ■■■
1736   Padding(
1737     const EdgeInsets.only(left: 12),
1738     Text(
1739       ■${finalPrice.toStringAsFixed(2)}',
1740       const TextStyle(
1741         14,
1742         Colors.black87,
1743       ),
1744     ),
1745   ),
1746   // ■■■■
1747   Padding(
1748     const EdgeInsets.only(left: 12, top: 4),
1749     Text(
1750       $dealTime',
```

[illegible]


```
1851 onTap,
1852 Container(
1853   const EdgeInsets.symmetric(horizontal: 24, vertical: 12),
1854   const EdgeInsets.symmetric(vertical: 8),
1855   BoxDecoration(
1856     isKasec
1857       ? const LinearGradient(
1858         Alignment.topLeft,
1859         Alignment.bottomRight,
1860         [Color(0xFFFFF5F5), Color(0xFFFFED7D7)],
1861       )
1862     const LinearGradient(
1863       Alignment.topLeft,
1864       Alignment.bottomRight,
1865       [Color(0xFFF0FFF4), Color(0xFFC6F6D5)],
1866     ),
1867     Border.all(
1868       isKasec ? const Color(0xFFE53E3E) : const Color(0xFF38A169),
1869       2,
1870     ),
1871     BorderRadius.circular(16),
1872     [
1873       BoxShadow(
1874         Colors.black.withValues(alpha: 0.1),
1875         0,
1876         4,
1877         const Offset(0, 2),
1878       ),
1879     ],
1880   ),
1881 ],
1882 ),
1883 Row(
1884   [
1885     // ■■■■
1886     Icon(
1887       Icons.flash_on,
1888       24,
1889       isKasec
1890         ? const Color(0xFFE53E3E)
1891       const Color(0xFF38A169),
1892     ),
1893     12),
1894     // ■■■■
1895     Expanded(
1896       Text(
1897         statusText,
1898         TextStyle(
1899           16,
1900           FontWeight.w600,
```

1925 /// ■■■■■■

1926 ///

```
1951         return ImageMessage(
1952     message,
1953     onTap,
1954     onLongPress,
1955     );
1956     case ChatMessageType.auctionBid:
1957     message, onTap: onTap);
1958     case ChatMessageType.auctionStart:
1959     message, onTap: onTap);
1960     case ChatMessageType.auctionEnd:
1961     message, onTap: onTap);
1962     case ChatMessageType.auctionDeal:
1963     message, onTap: onTap);
1964     case ChatMessageType.kasecStatusChanged:
1965     message, onTap: onTap);
1966     case ChatMessageType.receipt:
1967     case ChatMessageType.banUser:
1968     case ChatMessageType.backout:
1969     message);
1970     case ChatMessageType.welcome:
1971     message);
1972     case null:
1973     default:
1974         // ■■■■■■■■
1975         return TextMessage(
1976     ChatMessage(
1977     message.id,
1978     ChatMessageType.text,
1979     message.status,
1980     message.chan,
1981     message.from,
1982
1983
1984     message.fromName,
1985     message.fromAdmin,
1986     message.fromTag,
1987     message.tagClass,
1988     message.avatar,
1989     message.to,
1990     message.time,
1991     message.msg ?? '■■■■■■',
1992     message.payload,
1993     message.receipt,
1994     message.sequenceNumber,
1995     ),
1996     onTap,
1997     onLongPress,
1998     );
1999     }
2000 }
```



```
2001  }
2002  import 'package:cached_network_image/cached_network_image.dart';
2003  import 'package:flutter/material.dart';
2004  import 'package:flutter_html/flutter_html.dart';
2005  import 'package:molitao_app/data/models/chat_message_model.dart';
2006
2007  /// ■■■■■■■■
2008  /// ■■ UniApp AuctionStartMessage.vue ■■
2009  2px solid #ef4444
2010  #fff5f5
2011  "■■■■■"
2012  ■■■■■■■■HTML■
2013  class AuctionStartMessage extends StatelessWidget {
2014    final ChatMessage message;
2015    final VoidCallback? onTap;
2016
2017    const AuctionStartMessage({super.key, required this.message, this.onTap});
2018
2019    @override
2020    Widget build(BuildContext context) {
2021      String name = '';
2022      String description = '';
2023
2024      if (message.payload != null) {
2025        if (message.payload is Map<String, dynamic>) {
2026          final payload = message.payload as Map<String, dynamic>;
2027          name = payload['name'] ?? '';
2028          description = payload['description'] ?? '';
2029        }
2030      }
2031
2032      return GestureDetector(
2033        onTap,
2034
2035        Container(
2036          const EdgeInsets.symmetric(vertical: 8),
2037          BoxDecoration(
2038            Border.all(color: const Color(0xffEF4444), width: 2),
2039            const Color(0xffffF5F5),
2040            BorderRadius.circular(8),
2041          ),
2042          Stack(
2043            Clip.none,
2044            [
2045              // ■■■■■■
2046              Positioned(
2047                0,
2048                0,
2049                Container(
```

```
2051 const EdgeInsets.symmetric(horizontal: 8, vertical: 4),
2052 const BoxDecoration(
2053   Color(0xffEF4444),
2054   BorderRadius.only(
2055     Radius.circular(6),
2056     Radius.circular(8),
2057   ),
2058 ),
2059 const Text(
2060   '■■■■',
2061   TextStyle(
2062     Colors.white,
2063     12,
2064     FontWeight.bold,
2065   ),
2066 ),
2067 ),
2068 ),
2069 // ■■■■
2070 Container(
2071   const BoxConstraints(minWidth: 200, maxWidth: 350),
2072   const EdgeInsets.fromLTRB(16, 28, 16, 12),
2073   Column(
2074     CrossAxisAlignment.start,
2075     [
2076       Text(
2077         $name',
2078         const TextStyle(
2079           14,
2080           Colors.black87,
2081           FontWeight.w500,
2082         ),
2083       ),
2084       8),
2085       // ■■■■ description HTML■■■■■■■■■■
2086     ],
2087   ),
2088   Html(
2089     description,
2090     {
2091       Style(
2092         Margins.zero,
2093         HtmlPaddings.zero,
2094       ),
2095       Style(width: Width(double.infinity)),
2096       Style(
2097         FontSize(14),
2098         Colors.black87,
2099       ),
2100       Style(
```

■43■

[illegible]

```
2201     this.padding,
2202     this.margin,
2203     this.onTap,
2204     this.onLongPress,
2205   });
2206
2207   @override
2208   Widget build(BuildContext context) {
2209     return Container(
2210       margin ?? const EdgeInsets.symmetric(horizontal: 12, vertical: 4),
2211       Align(
2212         _getAlignment(),
2213         GestureDetector(
2214           onTap,
2215           onLongPress,
2216           Container(
2217             BoxConstraints(
2218               MediaQuery.of(context).size.width * 0.75,
2219             ),
2220             padding:
2221               padding ??
2222               12, vertical: 10),
2223             BoxDecoration(
2224               _getBackgroundColor(),
2225               _getBorderRadius(),
2226               [
2227                 BoxShadow(
2228                   Colors.black.withOpacity(0.05),
2229                   4,
2230                   const Offset(0, 1),
2231                 ),
2232               ],
2233             ),
2234             child,
2235           ),
2236         ),
2237       ),
2238     );
2239   }
2240
2241   Alignment _getAlignment() {
2242
2243
2244     switch (position) {
2245       case MessagePosition.left:
2246         return Alignment.centerLeft;
2247       case MessagePosition.right:
2248         return Alignment.centerRight;
2249       case MessagePosition.center:
2250         return Alignment.center;
```

■46■

```

2301
2302     const MessageWithAvatar({
2303         super.key,
2304         required this.child,
2305         this.avatarUrl,
2306         this.userName,
2307         this.isSelf = false,
2308         this.onAvatarTap,
2309         this.onMessageTap,
2310         this.onMessageLongPress,
2311         this.trailing,
2312     });
2313
2314     @override
2315     Widget build(BuildContext context) {
2316       if (isSelf) {
2317         // ████████████████████████
2318         return MessageBubble(
2319           MessagePosition.right,
2320           onMessageTap,
2321           onMessageLongPress,
2322           child,
2323         );
2324       }
2325
2326       // ████████████████████████
2327       return Container(
2328         const EdgeInsets.symmetric(horizontal: 12, vertical: 4),
2329         Row(
2330           CrossAxisAlignment.start,
2331           [
2332             // ■■
2333             onAvatarTap, child: _buildAvatar()),
2334           8),
2335           // ■■■■
2336           Expanded(
2337             Column(
2338               CrossAxisAlignment.start,
2339               [
2340                 // ■■■■
2341                 if (userName != null)
2342                   Padding(
2343                     const EdgeInsets.only(bottom: 4),
2344                     Text(
2345                       userName!,
2346
2347                       TextStyle(
2348                         12,
2349                         Colors.grey.shade600,

```

[illegible]


```
2401     Widget build(BuildContext context) {
2402         return MessageBubble(
2403             MessagePosition.center,
2404             const EdgeInsets.symmetric(horizontal: 16, vertical: 6),
2405             Text(
2406                 text,
2407                 TextStyle(fontSize: 12, color: Colors.grey.shade600),
2408                 TextAlign.center,
2409             ),
2410         );
2411     }
2412 }
2413 import 'package:flutter/material.dart';
2414 import 'package:cached_network_image/cached_network_image.dart';
2415 import '../data/models/chat_message_model.dart';
2416 import 'message_bubble.dart';
2417
2418 /// ■■■■■■
2419 ///
2420 /// ■■■■■■■■■■
2421 /// - ■■■■■■
2422 /// - ■■■■■■
2423 /// - ■■■■■■
2424 /// - ■■■■
2425 class ImageMessage extends StatelessWidget {
2426     final ChatMessage message;
2427     final bool isSelf;
2428     final VoidCallback? onTap;
2429     final VoidCallback? onLongPress;
2430     final Function(String)? onImageTap;
2431
2432     // ■■■■■■■■
2433     static const double maxWidth = 200;
2434     static const double maxHeight = 200;
2435     static const double minWidth = 100;
2436     static const double minHeight = 100;
2437
2438     const ImageMessage({
2439         super.key,
2440         required this.message,
2441         this.isSelf = false,
2442         this.onTap,
2443         this.onLongPress,
2444         this.onImageTap,
2445     });
2446
2447     @override
2448     Widget build(BuildContext context) {
2449         final imageUrl = _getImageUrl();
2450
```

```
2451
2452     final imageSize = _getImageSize();
2453
2454     return GestureDetector(
2455   () {
2456       if (onImageTap != null && imageUrl != null) {
2457         onImageTap!(imageUrl);
2458       }
2459       onTap?.call();
2460     },
2461     onLongPress,
2462     ClipRRect(
2463       BorderRadius.circular(8),
2464       Container(
2465         BoxConstraints(
2466           maxWidth,
2467           maxHeight,
2468           minWidth,
2469           minHeight,
2470         ),
2471         imageSize.width,
2472         imageSize.height,
2473         imageUrl != null
2474           ? CachedNetworkImage(
2475             imageUrl,
2476             BoxFit.cover,
2477             (context, url) => Container(
2478               Colors.grey.shade200,
2479               const Center(
2480                 SizedBox(
2481                   24,
2482                   24,
2483                   CircularProgressIndicator(strokeWidth: 2),
2484                 ),
2485               ),
2486             ),
2487             (context, url, error) => Container(
2488               Colors.grey.shade200,
2489               Column(
2490                 MainAxisAlignment.center,
2491                 [
2492                   Icon(
2493                     Icons.broken_image,
2494                     Colors.grey.shade400,
2495                     32,
2496                   ),
2497                   4),
2498                   Text(
2499                     '■■■■',
2500                     TextStyle(
```

[illegible]

■52■

[illegible]

```
2651
2652     if (width <= 0 || height <= 0) {
2653         return const Size(minWidth, minHeight);
2654     }
2655
2656     // ■■■■■■
2657     double displayWidth = width;
2658
2659
2660     double displayHeight = height;
2661
2662     // ■■■■■■■■■■
2663     if (width > maxWidth || height > maxHeight) {
2664         final widthRatio = maxWidth / width;
2665         final heightRatio = maxHeight / height;
2666         heightRatio;
2667         displayWidth = width * ratio;
2668         displayHeight = height * ratio;
2669     }
2670
2671     // ■■■■■■
2672     if (displayWidth < minWidth) displayWidth = minWidth;
2673     if (displayHeight < minHeight) displayHeight = minHeight;
2674
2675     return Size(displayWidth, displayHeight);
2676 } catch (e) {
2677     return const Size(minWidth, minHeight);
2678 }
2679 }
2680 }
2681
2682 /// ■■■■■■■■
2683 class ImageMessageWithAvatar extends StatelessWidget {
2684     final ChatMessage message;
2685     final bool isSelf;
2686     final VoidCallback? onAvatarTap;
2687     final VoidCallback? onMessageTap;
2688     final VoidCallback? onMessageLongPress;
2689     final Function(String)? onImageTap;
2690
2691     const ImageMessageWithAvatar({
2692         super.key,
2693         required this.message,
2694         this.isSelf = false,
2695         this.onAvatarTap,
2696         this.onMessageTap,
2697         this.onMessageLongPress,
2698         this.onImageTap,
2699     });
2700
```

```

2701     @override
2702     Widget build(BuildContext context) {
2703         final imageMessage = ImageMessage(
2704             message,
2705             isSelf,
2706             onMessageTap,
2707             onMessageLongPress,
2708             onImageTap,
2709             );
2710
2711
2712
2713         if (isSelf) {
2714             return MessageBubble(
2715                 MessagePosition.right,
2716                 EdgeInsets.zero,
2717                 onMessageTap,
2718                 onMessageLongPress,
2719                 imageMessage,
2720                 );
2721         }
2722
2723         return MessageWithAvatar(
2724             _getAvatarUrl(),
2725             message.fromName,
2726             isSelf,
2727             onAvatarTap,
2728             imageMessage,
2729             );
2730     }
2731
2732     String? _getAvatarUrl() {
2733         if (message.avatar == null) return null;
2734         final avatar = message.avatar!;
2735         if (avatar.startsWith('http')) return avatar;
2736         return 'https://image.molitao.top/$avatar';
2737     }
2738 }
2739 import 'package:flutter/material.dart';
2740 import 'package:molitao_app/utils/emoji_decoder.dart';
2741
2742 /// ■■■■■■■■
2743 class EmojiPicker extends StatelessWidget {
2744     /// ■■■■■■■■
2745     final void Function(String emojiCode) onEmojiSelected;
2746
2747     /// ■■■■■■■■■■■■■■■■
2748     final bool showFavoriteTab;
2749
2750     const EmojiPicker({

```

```
2751     Key? key,
2752     required this.onEmojiSelected,
2753     this.showFavoriteTab = false,
2754     super(key: key);
2755
2756     @override
2757     Widget build(BuildContext context) {
2758       return Container(
2759         Colors.white,
2760         Column(
2761           MainAxisSize.min,
2762
2763
2764         [
2765           // ■■■
2766           _buildTabBar(),
2767
2768           // ■■■■
2769           _buildEmojiGrid(),
2770         ],
2771       ),
2772     );
2773   }
2774
2775   Widget _buildTabBar() {
2776     return Container(
2777       36,
2778       const BoxDecoration(
2779         Border(bottom: BorderSide(color: Color(0xFFECECEC), width: 1)),
2780       ),
2781       Row(
2782         [
2783           _buildTab('■■', true),
2784           if (showFavoriteTab) _buildTab('■■', false),
2785         ],
2786       ),
2787     );
2788   }
2789
2790   Widget _buildTab(String label, bool isActive) {
2791     return Container(
2792       const EdgeInsets.symmetric(horizontal: 16, vertical: 8),
2793       BoxDecoration(
2794         isActive
2795           ? const Border(
2796             BorderSide(color: Color(0xFFFF4835A), width: 2),
2797           )
2798       null,
2799     ),
2800     Text(
```



```
2801         label,
2802         TextStyle(
2803           14,
2804           isActive ? const Color(0xFFFF4835A) : const Color(0xFF666666),
2805           isActive ? FontWeight.w500 : FontWeight.normal,
2806         ),
2807       ),
2808     );
2809   }
2810
2811   Widget _buildEmojiGrid() {
2812     final emojiCodes = EmojiDecoder.emojiCodes;
2813
2814
2815
2816     return GridView.builder(
2817       const EdgeInsets.all(8),
2818       const SliverGridDelegateWithFixedCrossAxisCount(
2819         8,
2820         8,
2821         8,
2822       ),
2823       emojiCodes.length,
2824       (context, index) {
2825         final code = emojiCodes[index];
2826         final url = EmojiDecoder.getEmojiUrl(code);
2827
2828         return GestureDetector(
2829           () => onEmojiSelected(code),
2830           Container(
2831             BoxDecoration(
2832               Colors.grey.shade50,
2833               BorderRadius.circular(4),
2834             ),
2835           url != null
2836             ? ClipRRect(
2837               BorderRadius.circular(4),
2838               Image.network(
2839                 url,
2840                 28,
2841                 28,
2842                 BoxFit.contain,
2843                 (context, error, stackTrace) {
2844                   return Center(
2845                     Text(
2846                       code.replaceAll('[', ' ').replaceAll(']', ' '),
2847                       const TextStyle(fontSize: 10),
2848                     ),
2849                   );
2850                 },
```

```
2851         ),
2852     )
2853     Center(
2854     Text(code, style: const TextStyle(fontSize: 10)),
2855     ),
2856     ),
2857     );
2858     },
2859     );
2860     }
2861     }
2862     import 'package:flutter/material.dart';
2863     import 'package:flutter/services.dart';
2864     import 'package:cached_network_image/cached_network_image.dart';
2865     import '../data/models/user_model.dart';
2866
2867
2868
2869     /// ■■■■■■
2870     /// ■■■■■■■■■■QQ■■■■■■■
2871     class UserProfileDialog extends StatelessWidget {
2872         final UserDto user;
2873
2874         const UserProfileDialog({super.key, required this.user});
2875
2876         /// ■■■■■■■■
2877         static Future<void> show(BuildContext context, UserDto user) {
2878             return showDialog(
2879             context,
2880             (context) => UserProfileDialog(user: user),
2881             );
2882         }
2883
2884         @override
2885         Widget build(BuildContext context) {
2886             return Dialog(
2887             RoundedRectangleBorder(borderRadius: BorderRadius.circular(16)),
2888             Container(
2889             const EdgeInsets.all(20),
2890             const BoxConstraints(maxWidth: 320),
2891             Column(
2892             MainAxisSize.min,
2893             [
2894                 // ■■■■■■ + ■■
2895                 Row(
2896                 [
2897                     // ■■
2898                     _buildAvatar(),
2899                 ],
2900                 // ■■
```

```
2901             Expanded(
2902   Text(
2903             user.name ?? user.fullName ?? '■■',
2904   const TextStyle(
2905     18,
2906     FontWeight.bold,
2907           ),
2908           ),
2909           ),
2910     ],
2911   ),
2912   20),
2913   1),
2914   16),
2915
2916           // ■■■■
2917   _buildInfoRow(
2918
2919           context,
2920   '■■■■',
2921   user.id?.toString() ?? '',
2922   true,
2923           ),
2924
2925           // QQ
2926   if (user.qq != null && user.qq!.isNotEmpty)
2927     _buildInfoRow(
2928       context,
2929   'QQ',
2930   user.qq!,
2931   true,
2932           ),
2933
2934           // ■■
2935   if (user.wx != null && user.wx!.isNotEmpty)
2936     _buildInfoRow(
2937       context,
2938   '■■',
2939   user.wx!,
2940   true,
2941           ),
2942
2943   20),
2944
2945           // ■■■■
2946   SizedBox(
2947     double.infinity,
2948     TextButton(
2949       () => Navigator.pop(context),
```

```
2951   TextButton.styleFrom(
2952     const Color(0xFFf4835a),
2953     Colors.white,
2954     const EdgeInsets.symmetric(vertical: 12),
2955     RoundedRectangleBorder(
2956       BorderRadius.circular(8),
2957     ),
2958   ),
2959   const Text('■■'),
2960 ),
2961 ),
2962 ],
2963 ),
2964 ),
2965 );
2966 }
2967
2968 /// ■■■■
2969 Widget _buildAvatar() {
2970
2971   String? avatarUrl;
2972   if (user.headImgUrl != null) {
2973     final headImg = user.headImgUrl!;
2974     if (headImg.startsWith('http')) {
2975       avatarUrl = headImg;
2976     } else {
2977       avatarUrl = 'https://image.molitao.top/$headImg';
2978     }
2979   }
2980 }
2981
2982 if (avatarUrl == null || avatarUrl.isEmpty) {
2983   // ■■■■
2984   return Container(
2985     64,
2986     64,
2987     BoxDecoration(
2988       const Color(0xFFf4835a),
2989       BoxShape.circle,
2990     ),
2991     Center(
2992       Text(
2993         _getAvatarText(user.name ?? user.fullName ?? '■■'),
2994         const TextStyle(
2995           Colors.white,
2996           20,
2997           FontWeight.bold,
2998         ),
2999       ),
3000     ),
```

```
3001         );
3002     }
3003
3004     return ClipRRect(
3005       BorderRadius.circular(8),
3006       CachedNetworkImage(
3007         avatarUrl,
3008         64,
3009         64,
3010         BoxFit.cover,
3011         (context, url) => Container(
3012           64,
3013           64,
3014           Colors.grey.shade200,
3015           const Center(child: CircularProgressIndicator(strokeWidth: 2)),
3016         ),
3017         (context, url, error) => Container(
3018           64,
3019           64,
3020           BoxDecoration(
3021             const Color(0xFFf4835a),
3022           ),
3023           BoxShape.circle,
3024         ),
3025         Center(
3026           Text(
3027             _getAvatarText(user.name ?? user.fullName ?? '■■■'),
3028             const TextStyle(
3029               Colors.white,
3030               20,
3031               FontWeight.bold,
3032             ),
3033           ),
3034         ),
3035       ),
3036     ),
3037   );
3038 }
3039
3040
3041 /// ■■■■■■
3042 Widget _buildInfoRow(
3043   BuildContext context, {
3044   required String label,
3045   required String value,
3046   bool copyable = false,
3047 }) {
3048   return Padding(
3049     const EdgeInsets.symmetric(vertical: 8),
3050     Row(
```

```
3051 [
3052     SizedBox(
3053       70,
3054       Text(
3055         label,
3056         TextStyle(color: Colors.grey.shade600, fontSize: 14),
3057       ),
3058     ),
3059     Expanded(
3060       Row(
3061         [
3062           Expanded(
3063             Text(value, style: const TextStyle(fontSize: 14)),
3064           ),
3065           if (copyable)
3066             IconButton(
3067               Icon(
3068                 Icons.copy,
3069                 18,
3070                 Colors.grey.shade500,
3071               ),
3072               () => _copyToClipboard(context, value),
3073             ),
3074         ],
3075       ),
3076       const BoxConstraints(),
3077     ),
3078   ],
3079 ),
3080 ),
3081 ],
3082 ),
3083 );
3084 }
3085
3086 /// ■■■■■■
3087 void _copyToClipboard(BuildContext context, String text) {
3088   text));
3089   ScaffoldMessenger.of(context).showSnackBar(
3090     const SnackBar(
3091       Text('■■■■■■■■'),
3092       Duration(seconds: 1),
3093       SnackBarBehavior.floating,
3094     ),
3095   );
3096 }
3097
3098 /// ■■■■■■
3099 String _getAvatarText(String name) {
3100   if (name.isEmpty) return '■■';
```

```

3101 // ██████████
3102 final runes = name.runes.toList();
3103 if (runes.length == 1) {
3104     return String.fromCharCode(runes.first);
3105 }
3106 return name.substring(0, 2);
3107 }
3108 }
3109 import 'package:flutter_riverpod/flutter_riverpod.dart';
3110 import '../domain/entities/my_count_entity.dart';
3111 
3112 final apiServiceProvider = Provider<ApiService>((ref) {
3113     return ApiService();
3114 });
3115 
3116 class ApiService {
3117     // Mock implementation for getMyCount
3118     Future<MyCountEntity> getMyCount() async {
3119         // Simulate API call delay
3120         500));
3121 
3122         // Return mock data - in real implementation, this would call the act...
3123         12, depositBalance: 100));
3124     }
3125 }

```